Todo Management Application

Revature — Full-Stack SDLC Project

System Documentation

The DJs

Prepared by
Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document Control

Revision History

Per IEEE 830 and ISO document-control practice, all revisions are recorded below.

Version	Date	Author	Description
0.1	10 June 2026	The DJs	LaTeX structure, task breakdown, ERD, Definition of Done, and Task Backlog
0.2	11 June 2026	The DJs	API Contract HTTP request/response trace documentation (payloads and status codes)
0.3	12 June 2026	The DJs	Phase 2 complete: wireframes, component design, architecture and security planning, LaTeX module refactor

Document Information

Field	Value
Document type	System Documentation (5-phase SDLC)
Organization	Revature
Project	Todo Management Application — Full-Stack SDLC
Team	The DJs
Authors	Jaehoon Song, Daniel Patience, Jacqueline Lainhart
Intended audience	Development team and Revature instructors (Trainers)
Module structure	Requirements / Behavioral / Structural / Team Work Records; `appendix/A01-A05`

Contents

1 Project Overview

1.1 Objective

Build a full-stack, secure Todo Management application as part of the Revature full-stack SDLC training program. The team moves from initial requirement analysis to a functional product demonstration, simulating a real-world Software Development Lifecycle (SDLC).

1.2 Technology Stack

Layer	Technology
Frontend	Angular
Backend	Java (Spring Boot, Web, Data JPA)
Database	SQLite
Build / Tools	Gradle, Git, GitHub

1.3 User Stories (Scope)

The following stories define *what* the system must deliver. Each phase builds on the prior phase toward a complete implementation.

1. **Registration:** As a new user, I can register an account.
2. **Authentication:** As a user, I can log in and log out securely.
3. **Task Management:** As a user, I can create, read, update, and delete primary Todo items.
4. **Organization:** As a user, I can create, read, update, and delete nested subtasks.

1.4 Project Roadmap

Phase	Name	Goal
1	Requirements Analysis	Deconstruct user stories; define schema and API contracts.
2	Design	Plan architecture, wireframes, and security flows.
3	Backend Development	Build REST API, security layer, and persistence.
4	Frontend Development	Build Angular UI and integrate with the API.
5	Presentation	Demonstrate working software and architecture.

Requirements Analysis Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-RAD-P1
Version: 0.3
Status: Draft
Issue date: June 15, 2026

2 Requirements Analysis

This portion decomposes the project user stories into granular technical tasks and defines the REST API contract—the agreed communication standard between the Angular frontend and the Spring Boot backend. A use case diagram summarizes each story as an actor–system interaction; the task breakdown and endpoint specifications establish what the system must expose before implementation begins.

2.1 Task Breakdown

2.1.1 Epic 1: Account Creation

User story: As a new user, I can register an account to start tracking my todo tasks.

Task ID	Description
AUTH-01	Define `User` entity fields (id, username, email, password); SQLite `users` table.
AUTH-02	Encrypt passwords before storage (persist hash only, never plaintext).
AUTH-03	Define validation rules (unique username/email, required fields).
AUTH-04	Specify registration endpoint: `POST /api/auth/register`.
AUTH-05	Document responses: HTTP 200 on success; HTTP 409 duplicate username/email; HTTP 400 on validation errors.

2.1.2 Epic 2: Authentication

User story: As a user, I can log in and log out to securely access my todo items.

Task ID	Description
AUTH-06	Verify username/password combination against the database on login.
AUTH-07	Specify login endpoint: `POST /api/auth/login`.
AUTH-08	Return HTTP 200 when request JSON matches stored credentials.
AUTH-09	Define session-based authentication for protected routes.
AUTH-10	Document logout behavior and HTTP 401 responses for unauthenticated access.

2.1.3 Epic 3: Task Management

User story: As a user, I can create, edit, and delete todo items to keep track of my work.

Task ID	Description
TODO-01	Define `Todo` entity (id, title, description, completed, user_id, timestamps).
TODO-02	Specify `POST /api/todos` — create todo.
TODO-03	Specify `GET /api/todos` — list todos for authenticated user.
TODO-04	Specify `GET /api/todos/{id}` — retrieve single todo.
TODO-05	Specify `PUT /api/todos/{id}` — update todo fields.
TODO-06	Specify `DELETE /api/todos/{id}` — delete todo (cascade subtasks).
TODO-07	Enforce ownership: users may only access their own todos.

2.1.4 Epic 4: Subtask Organization

User story: As a user, I can create, edit, and delete subtask items to better organize my primary tasks.

Task ID	Description
SUB-01	Define `Subtask` entity (id, title, completed, todo_id, timestamps).
SUB-02	Use `GET /api/todos/{id}` to retrieve the parent todo before subtask operations.
SUB-03	Specify `POST /api/todos/{id}/{subtask}` — create subtask.
SUB-04	Specify `PATCH /api/todos/{id}/{subtask}` — update subtask fields.
SUB-05	Specify `DELETE /api/todos/{id}/{subtask}` — delete subtask.
SUB-06	Validate parent todo exists, belongs to the authenticated user; document cascade delete when parent todo is removed.

2.2 API Contract

All endpoints must follow RESTful conventions. RESTful endpoints with JSON request/response bodies and expected HTTP status codes. Each exchange shows a raw HTTP request or response trace: status line, headers, a blank line, then the body. All paths are relative to the API base URL.

DELETE responses. A successful delete returns `HTTP/1.1 204 No Content` with no response body—the server has fulfilled the request and there is no representation to return (RFC 9110). Error cases (e.g. resource not found) return `4xx` with a JSON body as documented below.

2.2.1 Endpoint Summary

Method	Path	Purpose
POST	/api/auth/register	Register a new account
POST	/api/auth/login	Authenticate an existing account
POST	/api/todos	Create a todo
GET	/api/todos	List todos for the authenticated user
GET	/api/todos/{id}	Retrieve a single todo
PUT	/api/todos/{id}	Update a todo
DELETE	/api/todos/{id}	Delete a todo (cascade subtasks)
GET	/api/todos/{id}	Retrieve parent todo (before subtask operations)
POST	/api/todos/{id}/{subtask}	Create a subtask
PUT	/api/todos/{id}/{subtask}	Update a subtask
DELETE	/api/todos/{id}/{subtask}	Delete a subtask

2.2.2 Account Creation

Use the resource-creation flow template for this `POST` endpoint (Figure 1).

`POST /api/auth/register`

```
POST /api/auth/register HTTP/1.1
Content-Type: application/json

{
    "username": "String",
    "password": "String"
}
```

REQ →
← RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "accountId": Integer,
    "username": "String"
}

HTTP/1.1 409 Conflict
Content-Type: application/json

{
    "status": 409
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400
}
```

2.2.3 Authentication

Use the login/auth flow template for this `/auth` endpoint (Figure 2).

`POST /api/auth/login`

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
    "username": "String",
    "password": "String"
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "accountId": Integer,
    "username": "String"
}

HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
    "message": "String"
}
```

2.2.4 Task Management

Use the resource read, creation, and delete flow templates (Figures 3, 4, and 5).

`POST /api/todos`

```
POST /api/todos HTTP/1.1
Content-Type: application/json

{
    "title": "String",
    "description": "String",
    "completed": boolean
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400
}
```

`GET /api/todos`

```
GET /api/todos HTTP/1.1
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

[
    {
        "todoId": Integer,
        "accountId": Integer,
        "title": "String",
        "description": "String",
        "completed": boolean
    }
]
```

GET /api/todos/{id}

```
GET /api/todos/{id} HTTP/1.1
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404
}
```

PUT /api/todos/{id}

```
PUT /api/todos/{id} HTTP/1.1
Content-Type: application/json

{
    "title": "String",
    "description": "String",
    "completed": boolean
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404
}
```

DELETE /api/todos/{id}

Deletes the todo and cascades to its subtasks.

```
DELETE /api/todos/{id} HTTP/1.1
```

REQ ⇄ RESP

```
HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404
}
```

2.2.5 Subtask Organization

`GET /api/todos/{id}`

Retrieve the parent todo before subtask operations.

```
GET /api/todos/{id} HTTP/1.1
```

REQ ⟶ ⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}
```

Use the resource read, creation, and delete flow templates (Figures 6, 7, and 8).

`POST /api/todos/{id}/{subtask}`

```
POST /api/todos/{id}/{subtask} HTTP/1.1
Content-Type: application/json

{
    "title": "String",
    "description": "String",
    "completed": boolean
}
```

REQ ⟶ ⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400
}
```

`PUT /api/todos/{id}/{subtask}`

```
PUT /api/todos/{id}/{subtask} HTTP/1.1
Content-Type: application/json

{
    "title": "String",
    "description": "String",
    "completed": boolean
}
```

REQ ⟶ ⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "todoId": Integer,
    "accountId": Integer,
    "title": "String",
    "description": "String",
    "completed": boolean
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404
}
```

`DELETE /api/todos/{id}/{subtask}`

Removes one subtask from the parent todo. On success the server responds with `204 No Content` and an empty body; no row count or deleted entity is returned.

```
DELETE /api/todos/{id}/{subtask} HTTP/1.1
```

REQ ⟶
⟵ RESP

```
HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404
}
```

Behavioral Specification Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-BEH-P1
Version: 0.3
Status: Draft
Issue date: June 15, 2026

3 Behavioral Specification

This portion documents how actors and application layers interact during key operations. A state diagram captures the client login/session lifecycle; sequence diagrams model each user story as a flow of requests and responses, including success paths and error handling; UI wireframes map the same stories to screens, routes, and navigation so the frontend team can implement against a shared behavioral model.

3.1 Sequence Diagrams

The team modeled each user story as a request–response flow across five layers: Client/Requester, API/Interface Layer, Business Logic Layer, Data Access Layer, and Data Storage. Each diagram aligns with the REST endpoints in Section 2.2 and documents both success paths and error handling. Diagrams are grouped by epic below.

3.1.1 Epic 1: Account Creation

User story: As a new user, I can register an account to start tracking my todo tasks.
`POST /api/auth/register` — duplicate username/email returns `409`; validation errors return `400`. Figure 1 documents the resource creation flow: Scenario A persists a new account when the username is unique; Scenario B returns a conflict when the resource already exists.

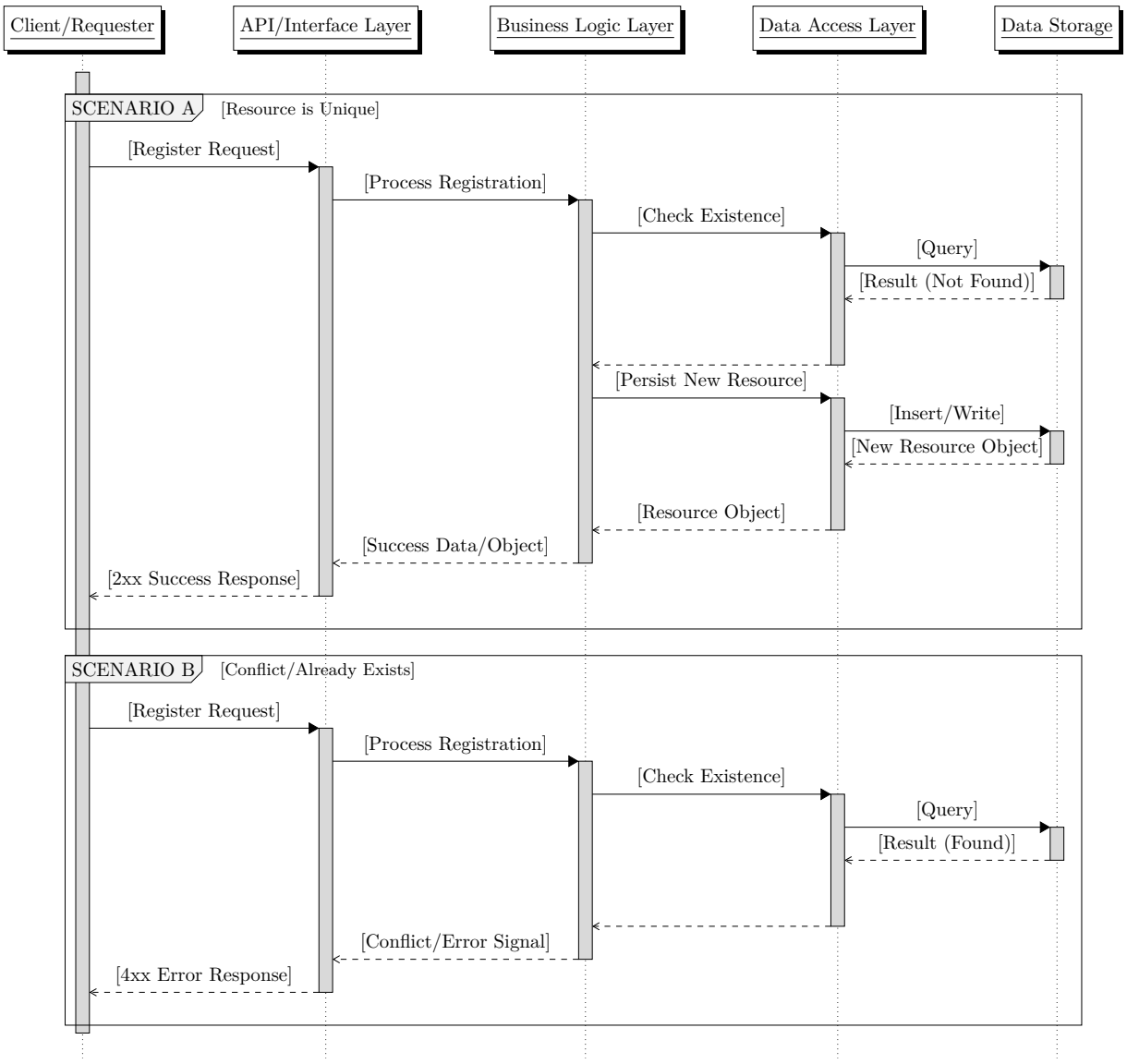


Figure 1: Account creation — `POST /api/auth/register`

3.1.2 Epic 2: Authentication

User story: As a user, I can log in and log out to securely access my todo items.
`POST /api/auth/login` — invalid credentials return `401`. Figure 2 documents the login/auth flow: Scenario A validates credentials and returns a token; Scenario B rejects invalid credentials.

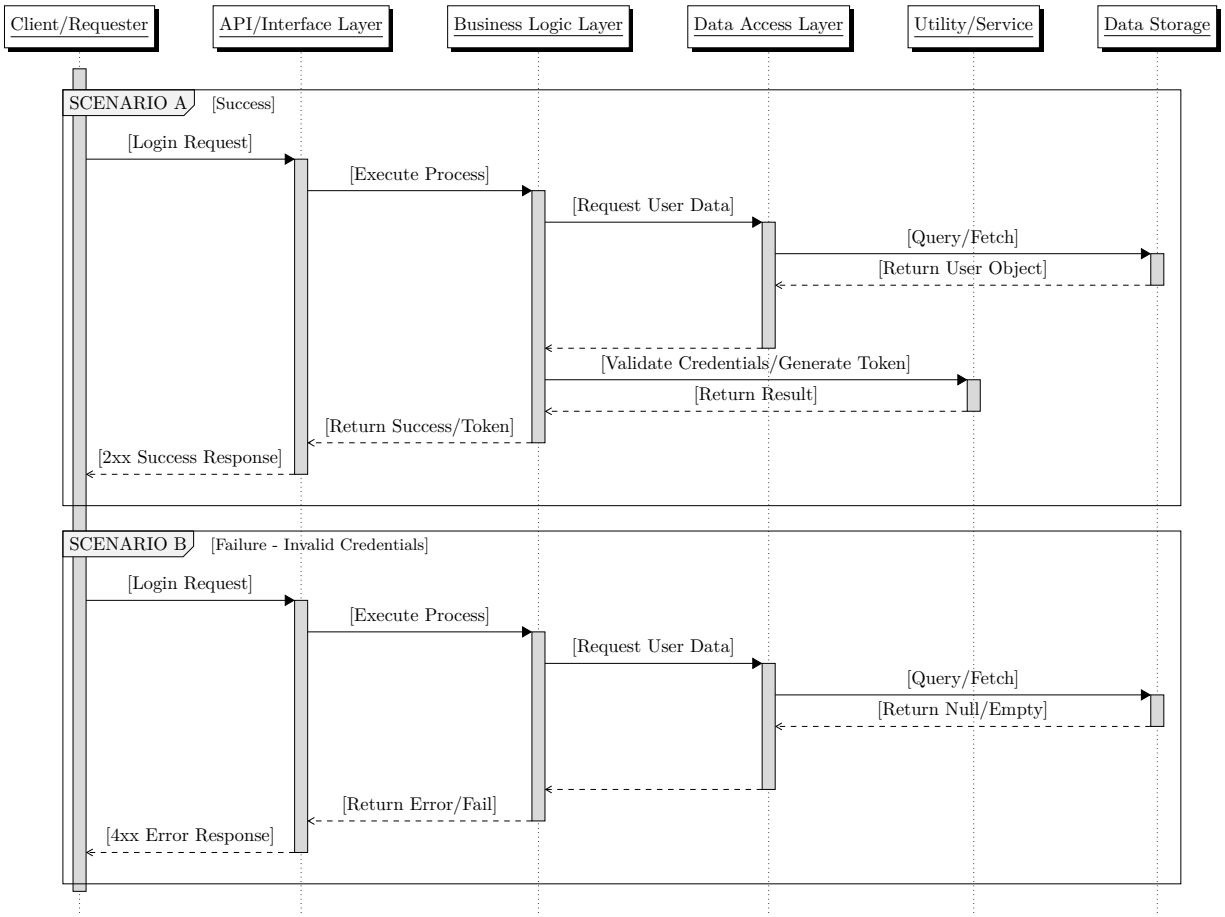


Figure 2: Authentication — `POST /api/auth/login`

3.1.3 Epic 3: Task Management

User story: As a user, I can create, edit, and delete todo items to keep track of my work.
`GET /api/todos` lists todos for the authenticated user; `POST /api/todos` and `PUT /api/todos/{id}` create or edit a todo; `DELETE /api/todos/{id}` removes a todo and cascades its subtasks. Validation errors return `400`; missing resources return `404`. Figure 3 documents the read flow; Figure 4 documents create and edit with success and null-result error paths; Figure 5 documents delete.

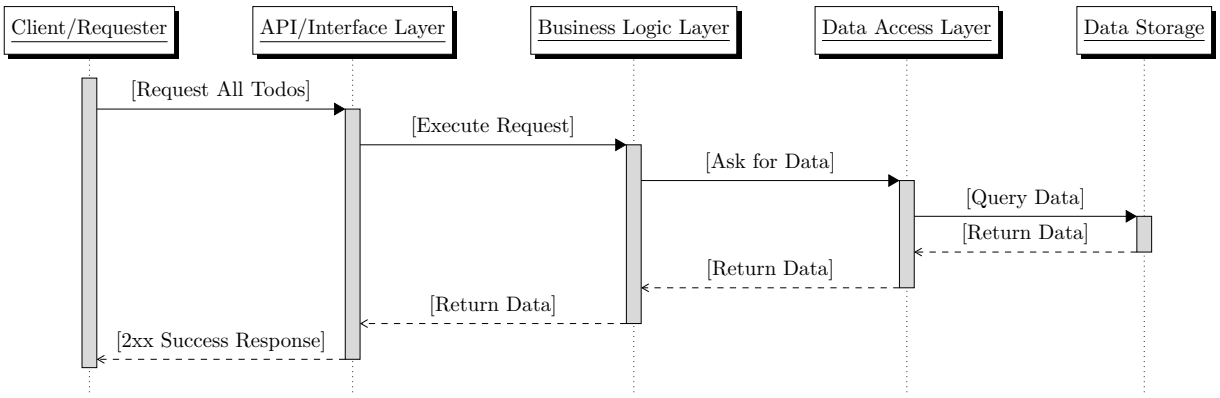


Figure 3: List todos — `GET /api/todos`

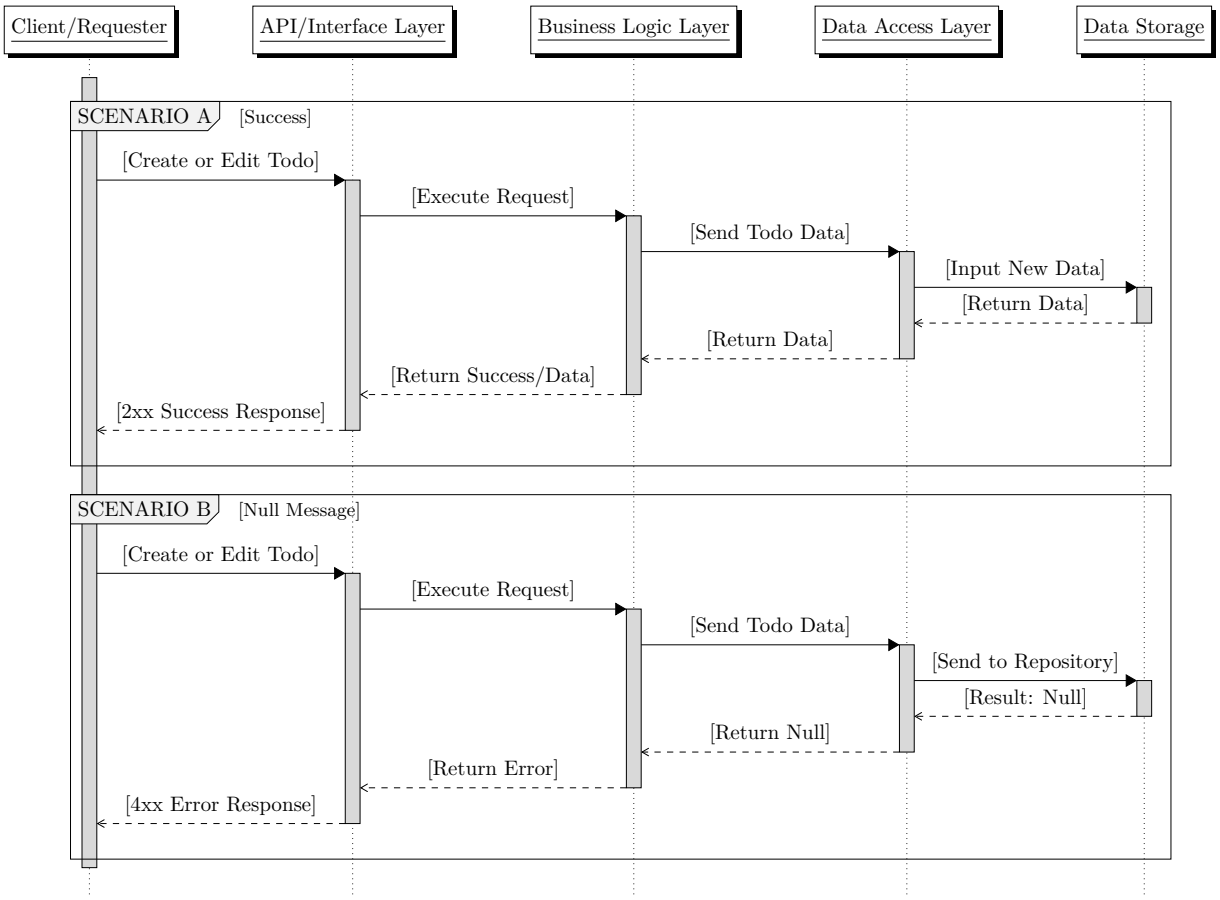


Figure 4: Create or edit todo — `POST /api/todos`, `PUT /api/todos/{id}`

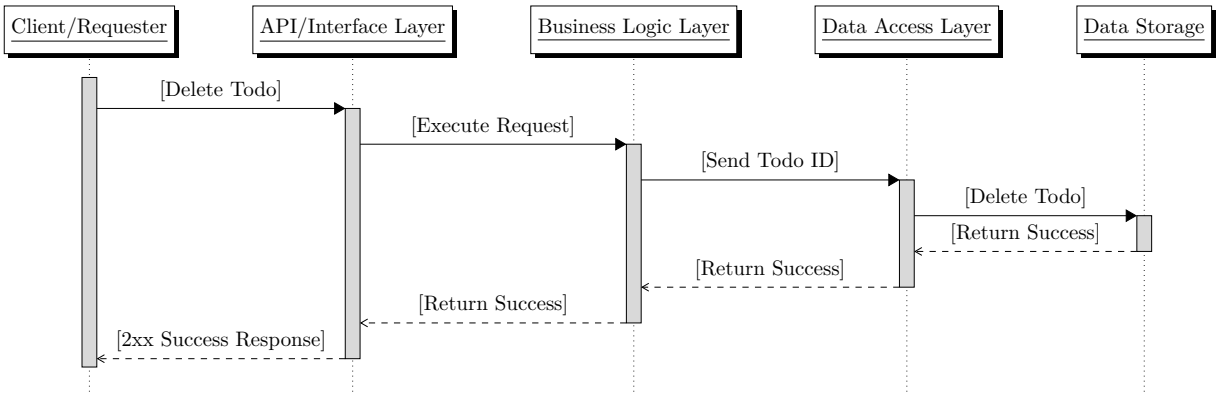


Figure 5: Delete todo — `DELETE /api/todos/{id}` (API contract: `204 No Content`)

3.1.4 Epic 4: Subtask Organization

User story: As a user, I can create, edit, and delete subtask items to better organize my primary tasks. `GET /api/todos/{id}` retrieves the parent todo before subtask operations; `POST /api/todos/{id}/{subtask}` and `PUT /api/todos/{id}/{subtask}` create or edit a subtask; `DELETE /api/todos/{id}/{subtask}` removes one subtask. The parent todo must exist and belong to the authenticated user. Figure 6 documents the read flow in parent-todo context; Figure 7 documents create and edit with success and null-result error paths; Figure 8 documents delete.

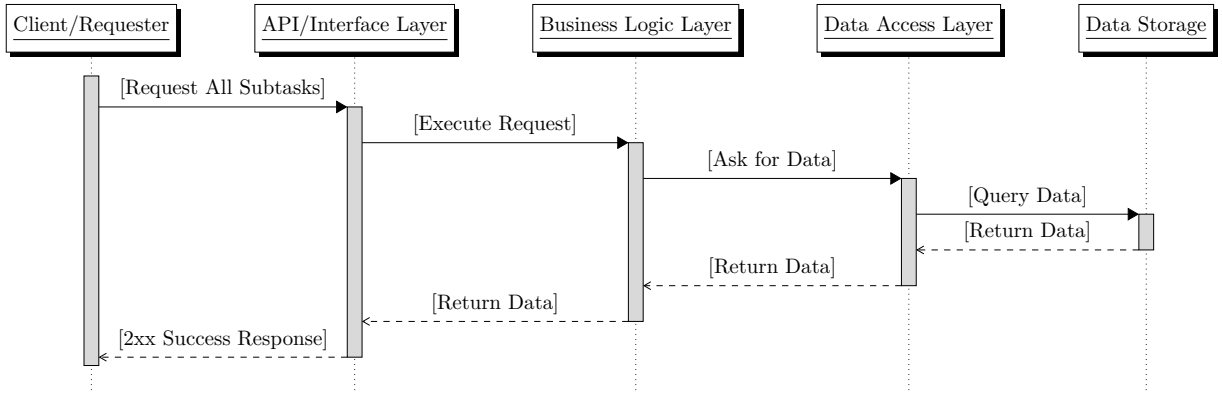


Figure 6: List subtasks — `GET /api/todos/{id}` (parent todo context)

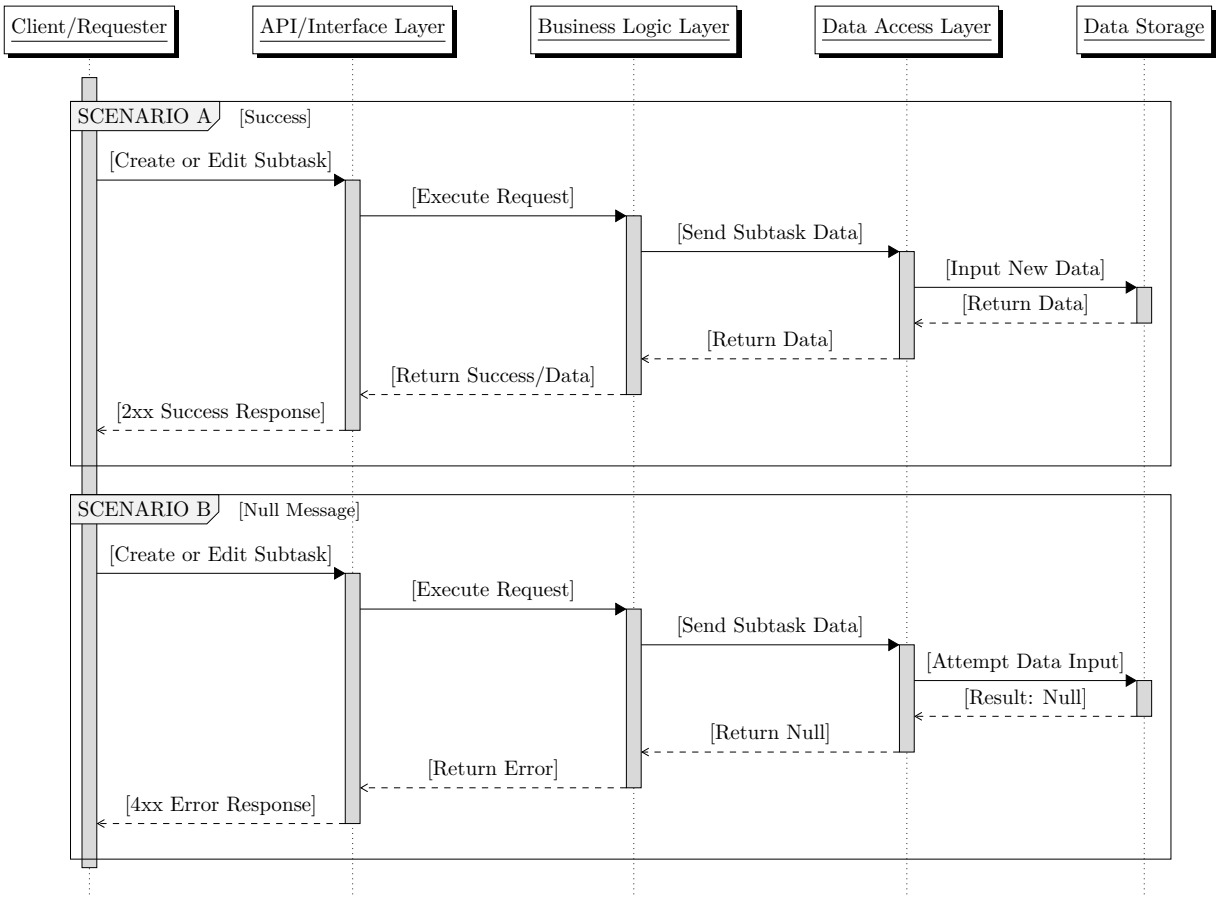


Figure 7: Create or edit subtask — `POST /api/todos/{id}/{subtask}`, `PUT /api/todos/{id}/{subtask}`

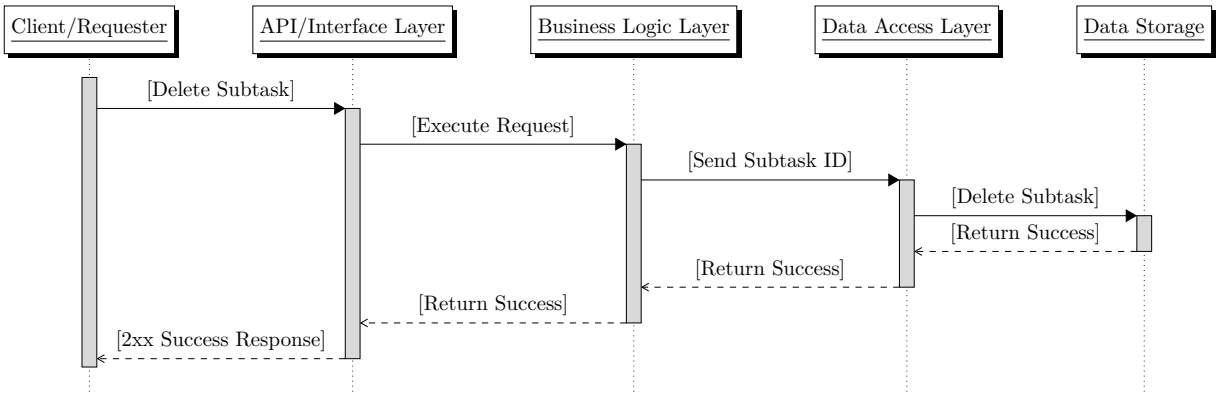


Figure 8: Delete subtask — `DELETE /api/todos/{id}/{subtask}` (API contract: `204 No Content`)

3.2 UI Wireframing

Low-fidelity wireframes define the primary screens and navigation paths for the Angular frontend. Each view maps to the user stories in Section 1; arrows in the interaction overview show how the client moves between routes after register, login, logout, and dashboard actions.

3.2.1 Screen Navigation

Figure 9 summarizes the application flow. Unauthenticated users land on the home (login) screen; **SIGN UP** routes to register and **LOG IN** or successful **CREATE ACCOUNT** routes to the dashboard. **LOG OUT** returns the user to login.

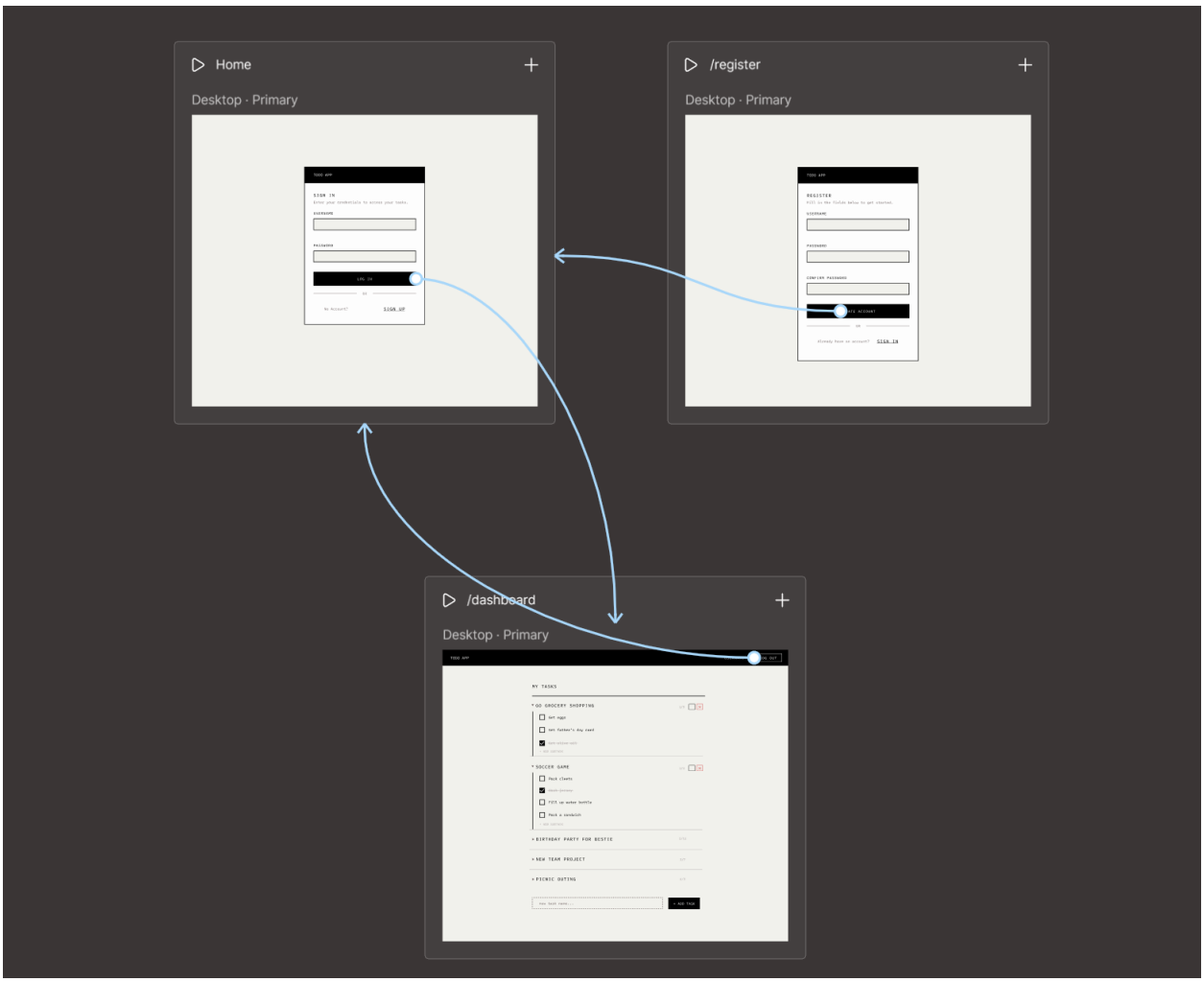
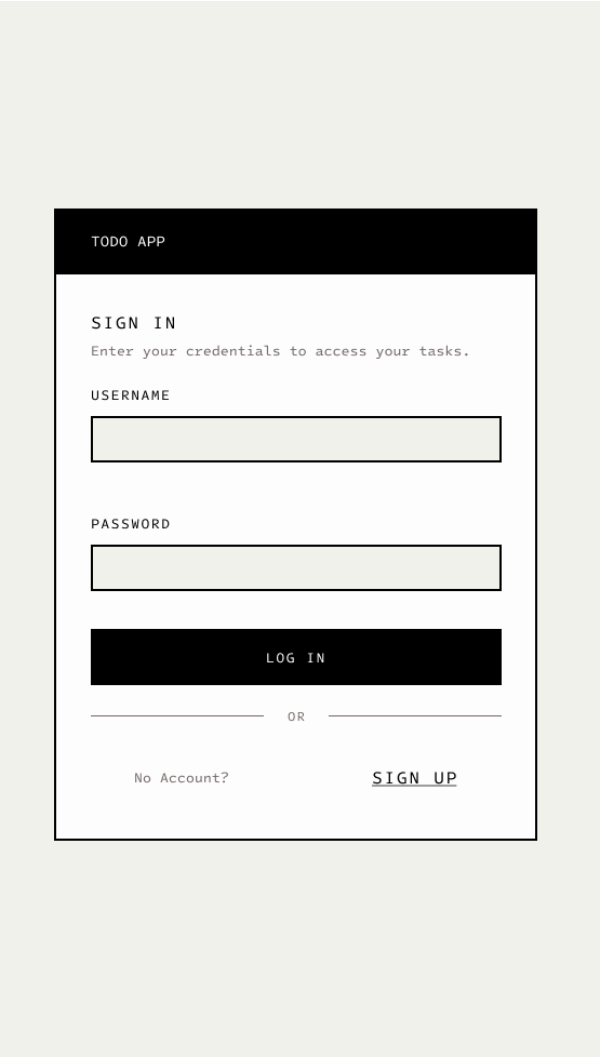


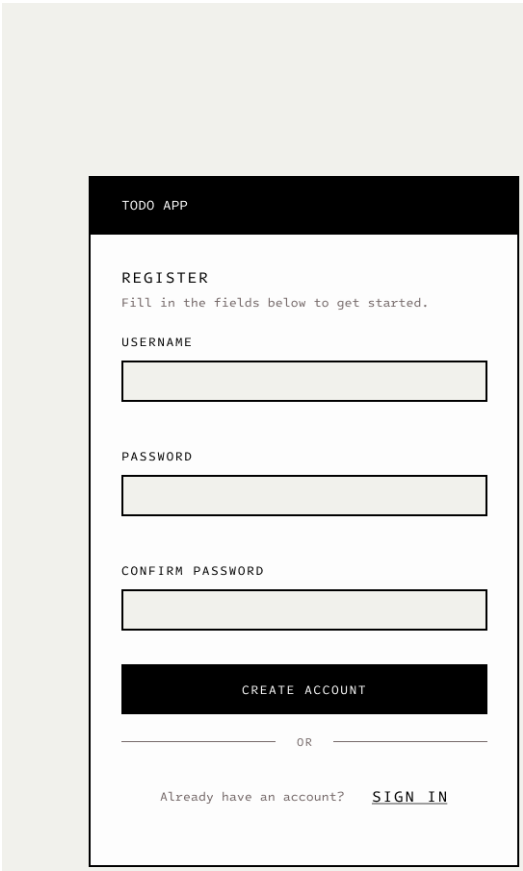
Figure 9: Screen navigation — home, register, and dashboard

3.2.2 Authentication Screens

Figures 10a and 10b cover Epic 1 (account creation) and Epic 2 (authentication). Both screens share the `TODO APP` header and a footer link to switch between sign-in and sign-up.



(a) Login — home route (`/`)



(b) Register — `/register`

3.2.3 Task Dashboard

Figure 11 covers Epic 3 (task management) and Epic 4 (subtask organization). The dashboard lists primary todos with expandable groups, completion progress (e.g. `1/3`), checkboxes, and delete controls. Users add top-level tasks via **+ ADD TASK** and nested items via **+ ADD SUBTASK**. The header shows the authenticated username and a **LOG OUT** action.

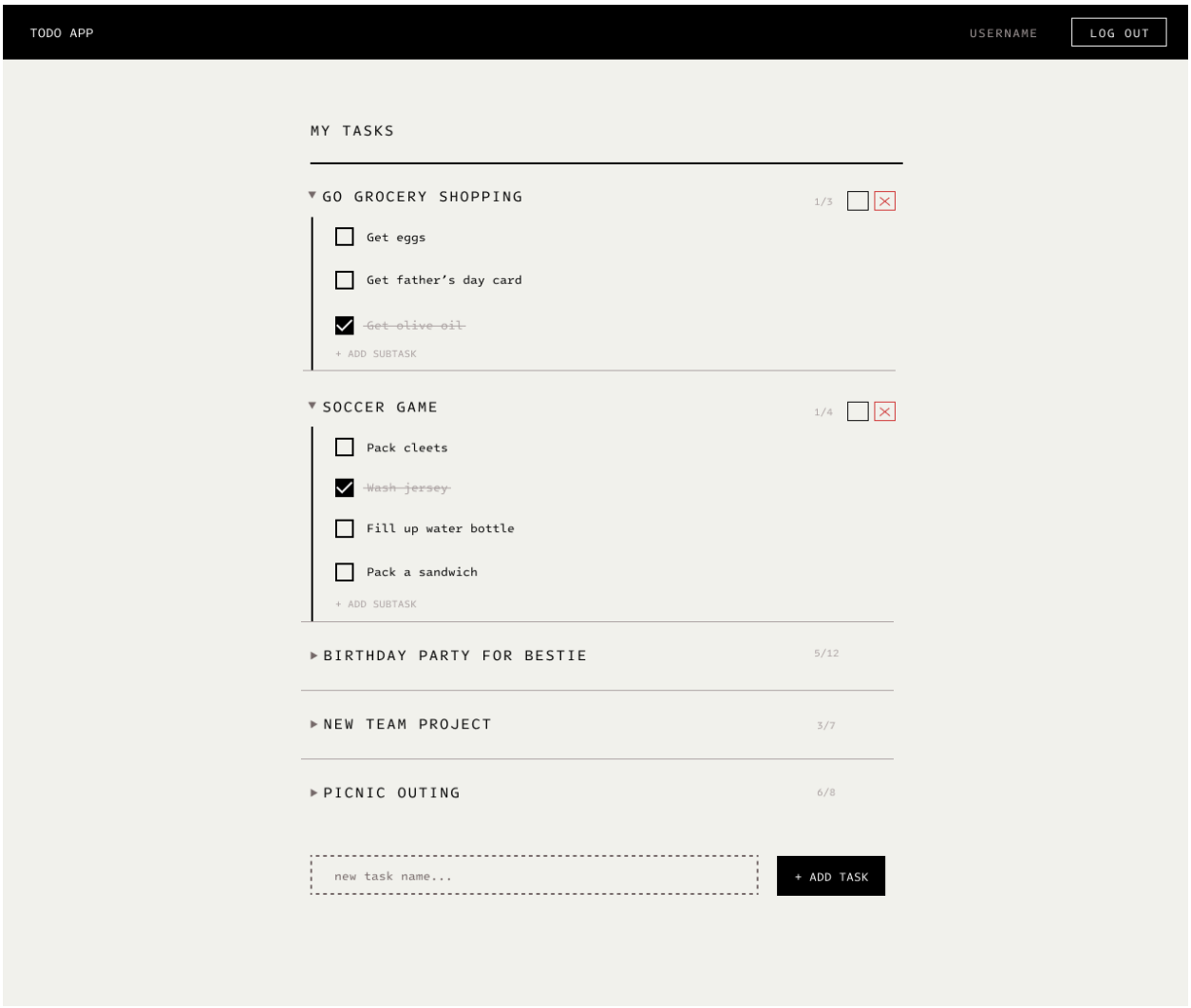


Figure 11: Dashboard — `/dashboard`

Structural Specification Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-STR-P1
Version: 0.3
Status: Draft
Issue date: June 15, 2026

4 Structural Specification

This portion defines the persistent data model and system structure for the Todo Management application. The component diagram, entity summary, Crow's Foot ERD, JPA class diagram, data dictionary, and integrity rules describe how the application is deployed and how `User`, `Todo`, and `Subtask` data is organized and constrained in SQLite.

4.1 Database Modeling

4.1.1 Entity Summary

Entity	Table	Purpose
User	`users`	Account credentials and identity
Todo	`todos`	Primary task items owned by a user
Subtask	`subtasks`	Nested items belonging to one todo

4.1.2 Entity Relationship Diagram

Figure 12 shows a compact **Crow's Foot** (IE) style ERD. Each `todo` references exactly one `user` (`user_id`); each `user` owns zero or more `todos`. Each `subtask` references exactly one `todo` (`todo_id`); each `todo` has zero or more `subtasks`.

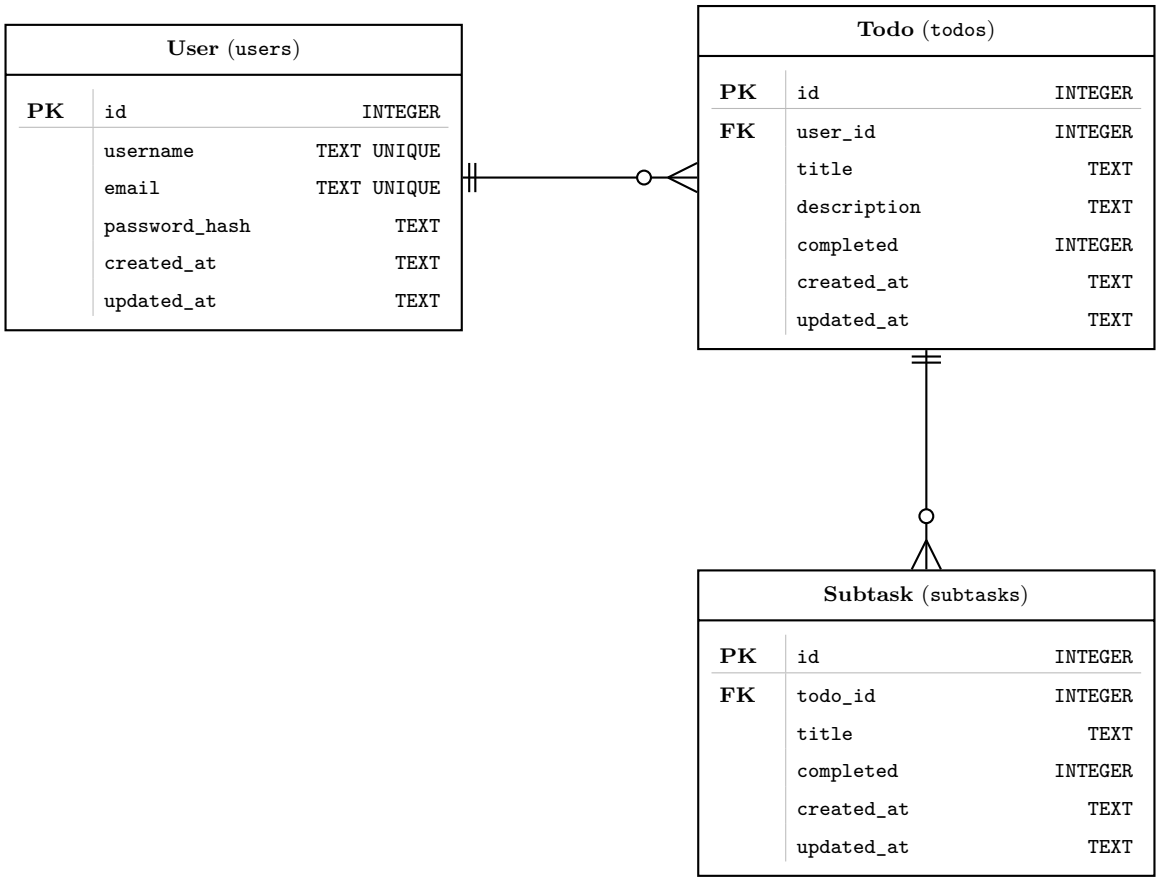


Figure 12: Crow's Foot ERD

4.1.3 Data Dictionary

The ERD above is the canonical model. Table 1 maps each logical entity to its SQLite columns, types, and constraints.

Table 1: SQLite data dictionary

Table	Column	Type	Constraints / Notes
users	id	INTEGER	PRIMARY KEY, AUTOINCREMENT
users	username	TEXT	NOT NULL, UNIQUE
users	email	TEXT	NOT NULL, UNIQUE
users	password_hash	TEXT	NOT NULL
users	created_at	TEXT	NOT NULL; ISO-8601 timestamp
users	updated_at	TEXT	NOT NULL; ISO-8601 timestamp
todos	id	INTEGER	PRIMARY KEY, AUTOINCREMENT
todos	user_id	INTEGER	NOT NULL; REFERENCES users(id), ON DELETE CASCADE
todos	title	TEXT	NOT NULL
todos	description	TEXT	Optional
todos	completed	INTEGER	NOT NULL, DEFAULT 0 (0 = false, 1 = true)
todos	created_at	TEXT	NOT NULL; ISO-8601 timestamp
todos	updated_at	TEXT	NOT NULL; ISO-8601 timestamp
subtasks	id	INTEGER	PRIMARY KEY, AUTOINCREMENT
subtasks	todo_id	INTEGER	NOT NULL; REFERENCES todos(id), ON DELETE CASCADE
subtasks	title	TEXT	NOT NULL
subtasks	completed	INTEGER	NOT NULL, DEFAULT 0
subtasks	created_at	TEXT	NOT NULL; ISO-8601 timestamp
subtasks	updated_at	TEXT	NOT NULL; ISO-8601 timestamp

4.1.4 Integrity Rules

- `users.username` and `users.email` must be unique.
- Every `todo` must reference a valid `user_id`.
- Every `subtask` must reference a valid `todo_id`.
- Deleting a user cascades to their todos and subtasks.
- Deleting a todo cascades to its subtasks.

Team Work Records

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-TWR-P1
Version: 0.3
Status: Draft
Issue date: June 15, 2026

5 Team Work Records

This portion records how the team measures completion and tracks ongoing work. The Definition of Done establishes criteria for finished deliverables; the task backlog summarizes session outcomes, backlog status, and the GitHub Issues workflow used to assign remaining items.

5.1 Definition of Done

The team established the following criteria on **10 June 2026**. A Phase 1 checkpoint or deliverable is considered *done* when all applicable items below are satisfied.

5.1.1 Documentation Criteria

- Content reflects decisions made in team discussion (see `discussions/` in the repository).
- Technical writing is merged into `docs/module/` (modular `.tex` files) and compiles without errors.
- Section owners verify accuracy before the item is marked complete in the checkpoint list.

5.1.2 Collaboration Criteria

- Work is committed on a feature branch and merged via pull request—not pushed directly to `main`.
- Discussion notes and LaTeX documentation may land on separate branches and are integrated independently.
- At least one teammate reviews the pull request before merge.

5.1.3 Checkpoint Exit Criteria

- **Task Breakdown** — All four epics documented with task IDs (Section 2.1).
- **Database Modeling** — ERD, data dictionary, and integrity rules agreed upon (Section 4.1).
- **API Contract** — Every endpoint from the task breakdown has request/response bodies and HTTP status codes (Section 2.2).
- **Task Backlog** — Remaining Phase 1 items tracked in GitHub Issues or Projects (Section 5.2).

5.1.4 Phase 1 Completion

As of 11 June 2026, all Phase 1 checkpoints are complete. The team may proceed to Phase 2 once all members have reviewed Sections 2.1–2.2.

5.1.5 Phase 2 Progress

Phase 2 is *in progress* as of 12 June 2026. A deliverable counts toward Phase 2 when it meets the documentation and collaboration criteria above and is recorded in the task backlog.

- **UI Wireframing** — Complete: Login, Register, Dashboard, and navigation flow (Section 3.2).
- **Component Design** — Complete: routes, component tree, and service layer agreed (Section 5.2).
- **LaTeX refactor** — Complete: modular `docs/module/` layout, section introductions, expanded sequence diagrams.
- **System Architecture** — Open: data-flow documentation not yet merged into the main document.
- **Security Planning** — Open: JJWT and auth-filter design pending.

5.2 Task Backlog

Use GitHub Issues and Github Project to track actionable items. Phase 1 completed **11 June 2026**; Phase 2 completed **12 June 2026**. Session summaries below record how each phase was closed out.

5.2.1 Session Summary — 10 June 2026

Activity	Outcome
LaTeX documentation initiated	Jaehoon Song set up the `docs/` structure and `main.tex` driver.
Team meeting — task breakdown	All members decomposed user stories into epics and task IDs; documented in Section 2.1.
Initial ERD	Jaehoon Song drafted the Crow's Foot ERD and SQLite data dictionary (Section 4.1).
Repository integration	Each member used feature branches to merge discussion notes and LaTeX updates separately into `main`.
Definition of Done	Team agreed on completion criteria (Section 5.1).
Task Backlog	Initial Phase 1 progress and GitHub workflow recorded (Section 5.2).

5.2.2 Session Summary — 11 June 2026

Activity	Outcome
API Contract	Daniel Patience and Jacqueline Lainhart drafted payloads and status codes; documented in Section 2.2.
HTTP request/response documentation	Jaehoon Song formatted API exchanges as HTTP traces (`httpexchange` layout); Section 2.2.
Phase 1 completion	All Phase 1 checkpoints satisfied; team cleared to begin Phase 2 (Section 5.1).

5.2.3 Session Summary — 12 June 2026

Activity	Outcome
LaTeX documentation refactor	Jaehoon Song reorganized content into `docs/module/`, added section introductions in `main.tex`, and expanded Epic 3/4 sequence diagrams (Section 3.1).
UI Wireframing	Login, Register, Dashboard, and screen-navigation wireframes added to `docs/Wireframes/` and documented in Section 3.2.
Component Design	Team agreed Angular routes (`/`, `/register`, `/dashboard`), component hierarchy (`AppComponent`, auth views, `TodoItemComponent` with nested subtasks), and services (`AuthService`, `TodoService`, `SubtaskService`, `AuthInterceptor`, `AuthGuard`).
Phase 2 completion	All Phase 2 checkpoints and deliverables satisfied; team cleared to begin Phase 3 (Section 5.1).

5.2.4 GitHub Workflow

1. Create a branch from `main` for the work item (discussion notes or documentation).
2. Commit changes locally; open a pull request when ready for review.
3. A teammate reviews and approves; merge into `main`.
4. Open or update a GitHub Issue for any remaining task IDs; link the issue to the pull request where applicable.

5.2.5 Backlog Status — Phase 1 (complete 11 June 2026)

Item	Status	Notes
Task Breakdown (Section 2.1)	Complete	Documented from team meeting discussion; 10 June 2026.
Database Modeling (Section 4.1)	Complete	Initial ERD and data dictionary; 10 June 2026.
API Contract (Section 2.2)	Complete	Payloads by Daniel Patience and Jacqueline Lainhart; HTTP req/res traces by Jaehoon Song; 11 June 2026.
Definition of Done (Section 5.1)	Complete	Criteria established; 10 June 2026.
GitHub Issues / Project board	Complete	Workflow recorded; issues to be opened as Phase 3 work is assigned.

5.2.6 Backlog Status — Phase 2 (complete 12 June 2026)

Item	Status	Notes
System Architecture (Appendix B.6)	Complete	Data flow documented in `discussions/phase-2-deliverables.md`; 12 June 2026.
Security Planning (Appendix B.7)	Complete	JJWT and auth approach agreed; 12 June 2026.
UI Wireframing (Section 3.2)	Complete	Four wireframes under `docs/Wireframes/`; 12 June 2026.
Component Design (Appendix B.9)	Complete	Routes, component tree, and services agreed; 12 June 2026.
LaTeX documentation refactor	Complete	Modular `docs/module/` structure and section intros; 12 June 2026.
API Specification (Section 2.2)	Complete	Finalized for implementation; 12 June 2026.
Database Schema (Section 4.1)	Complete	ERD and data dictionary signed off; 12 June 2026.

A Phase 1: Requirements Analysis

A.1 Objective

The goal of this phase is the technical decomposition of user stories. Before any code is written, the team must define the data structures, the API communication standards, and the criteria for successful completion.

A.2 Checkpoints

- ✓ **Task Breakdown:** Deconstruct user stories into granular technical tasks.
- ✓ **Database Modeling:** Design the relational schema (User, Todo, and Subtask entities).
- ✓ **API Contract:** Define RESTful endpoints and expected JSON request/response bodies.
- ✓ **Definition of Done (DoD):** Establish team-specific criteria for completed work.

A.3 Technical Reference: User Stories

Use these stories as the foundation for your task breakdown and API design:

1. **Account Creation:** As a new user, I can register an account to start tracking my todo tasks.
2. **Authentication:** As a new user, I can log in and out to securely access my todo items.
3. **Task Management:** As a user, I can create, edit, and delete todo items to keep track of my work.
4. **Subtask Organization:** As a user, I can create, edit, and delete subtask items to better organize my primary tasks.

A.4 Deliverables for Phase 1

By the end of this phase, your team should have a shared document or repository folder containing:

1. **Entity Relationship Diagram (ERD):** Visual or text-based representation of your SQLite schema.
2. **API Specification:** A list of endpoints (e.g., `POST /api/auth/register`, `GET /api/todos`) including expected payloads.
3. **Task Backlog:** A list of specific, actionable items ready to be assigned to team-members.
    - Use GitHub Issues and Github Project to facilitate this.

A.5 Moving to Phase 2

Once all Phase 1 checkboxes are marked as complete and the team agrees on the API contracts, proceed to **Phase 2: Design**.

B Phase 2: Design

B.1 Objective

The goal of this phase is architectural and interface planning. The team must define the data flow, security protocols, and user interface layouts. Successful completion of this phase ensures that the Backend and Frontend teams can work in parallel without integration conflicts.

B.2 Checkpoints

- ✓ **System Architecture:** Document the data flow (Angular → Spring Boot → SQLite).
- ☐ **Security Planning:** Familiarize yourself with JJWT.
- ✓ **UI Wireframing:** Map out the interface for authentication and task management (Section 3.2).
- ✓ **Component Design:** Loosely define the Angular component hierarchy and service structure (Section 5.2, 12 June 2026).

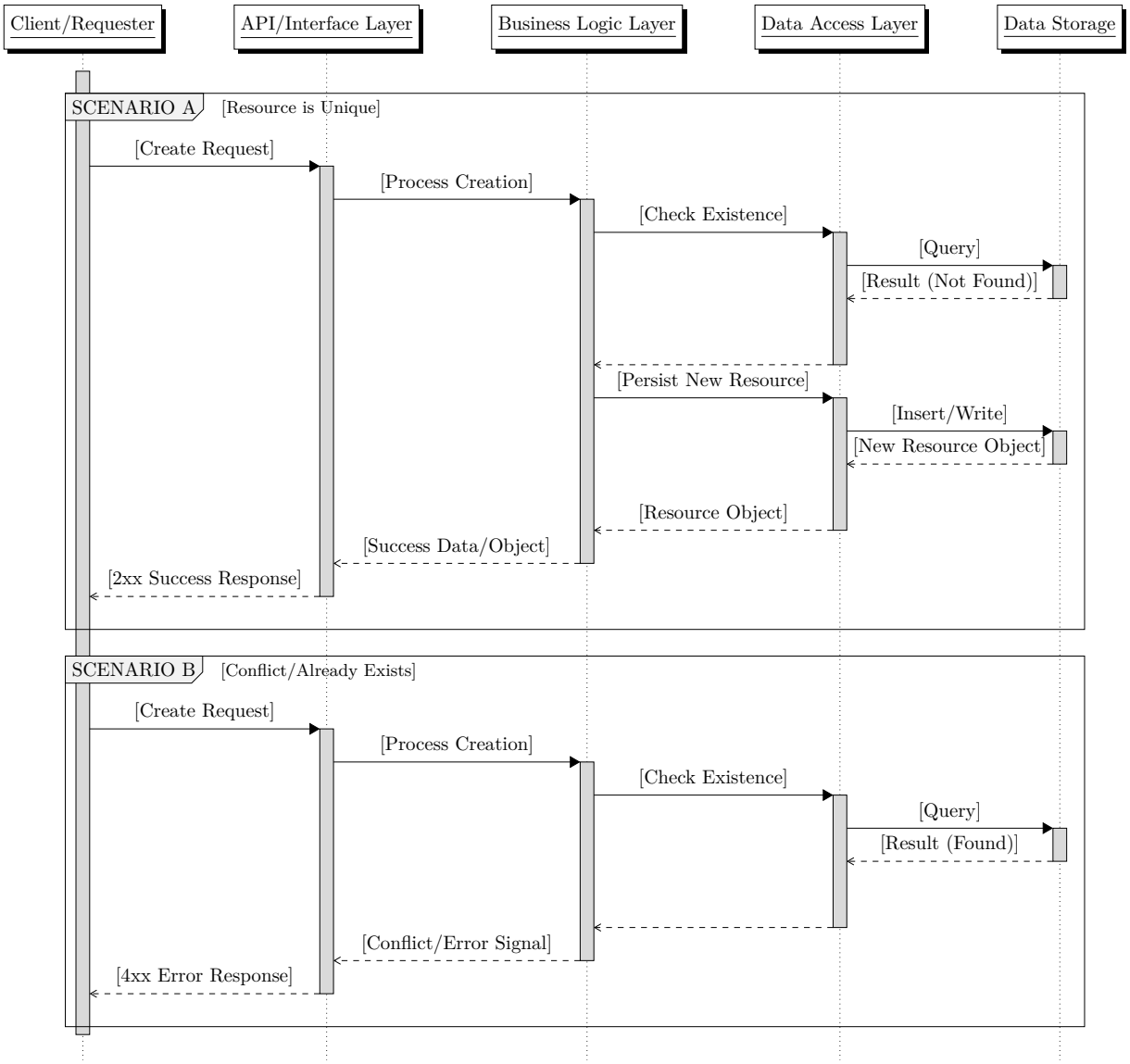
B.3 Technical Reference: Design Standards

B.3.1 API Design Principles

All endpoints must follow RESTful conventions. Use the following templates for your design documentation:

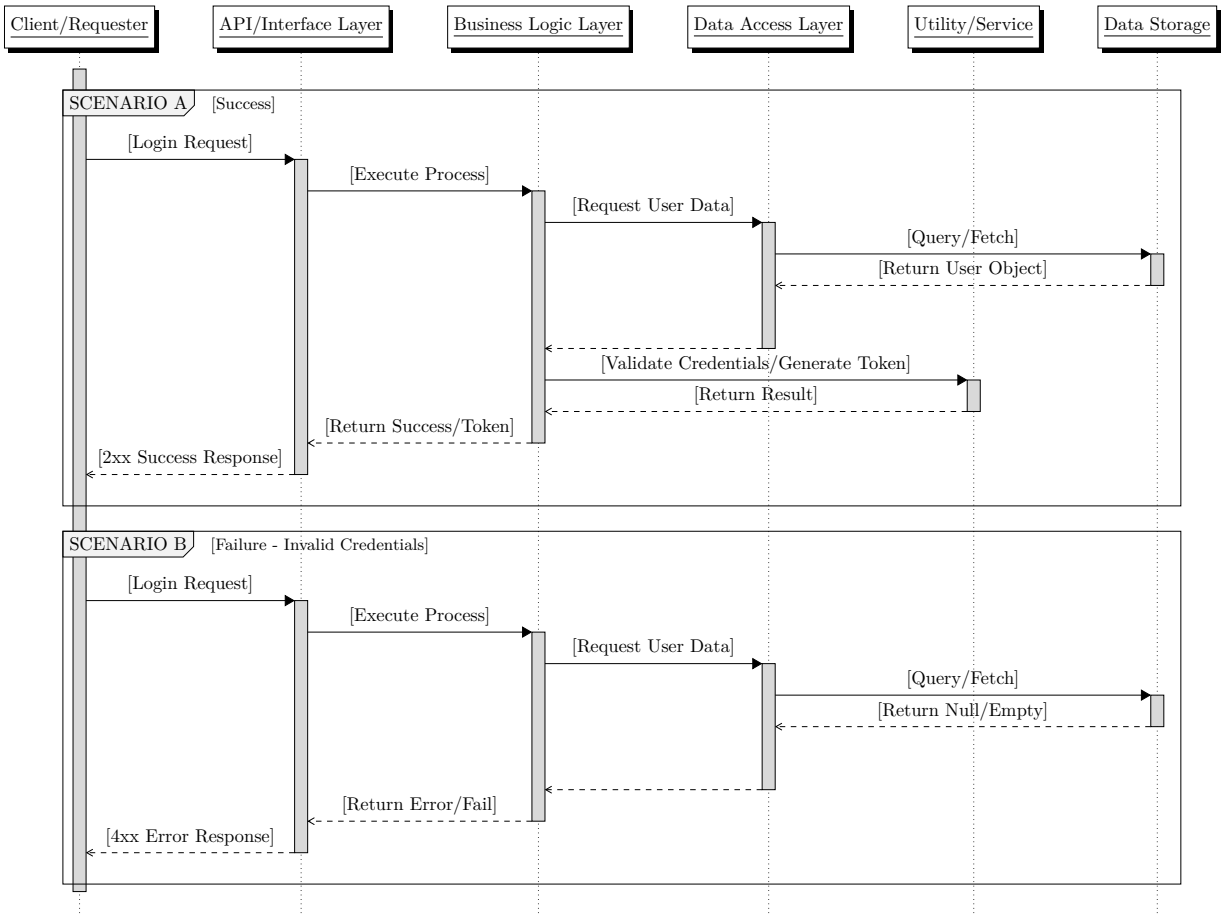
B.3.2 Resource Creation Flow (Template)

Use this logic to design your `POST` endpoints to ensure consistent error handling (e.g., handling duplicate entries).



B.3.3 Login/Auth Flow (Template)

Use this logic to design your `/auth` endpoints to ensure secure credential validation.



B.4 Deliverables for Phase 2

By the end of this phase, your team should have:

1. **API Specification:** A completed document defining all endpoints, request bodies, and response codes.
2. **Wireframes:** Visual mockups of the Login, Register, and Dashboard screens.
3. **Database Schema:** A final relational model (ERD) ready for implementation.

B.5 Moving to Phase 3

Once the API contracts and UI wireframes are finalized and agreed upon by all team members, proceed to **Phase 3: Backend Development**.

B.6 System Architecture

Document the data flow (Angular → Spring Boot → SQLite). See deliverable checklist in Section B.

B.7 Security Planning

Familiarize yourself with JJWT. See deliverable checklist in Section B.

B.8 UI Wireframing

Map out the interface for authentication and task management. See deliverable checklist in Section B.

B.9 Component Design

Loosely define the Angular component hierarchy and service structure. See deliverable checklist in Section B.

B.10 API Specification

Complete the document defining all endpoints, request bodies, and response codes. See deliverable checklist in Section B.

B.11 Database Schema

Finalize the relational model (ERD) ready for implementation. See deliverable checklist in Section B.

C Phase 3: Backend Development

C.1 Objective

The goal of this phase is to build a robust, secure REST API. This phase focuses on the "engine" of the application: data persistence, business logic, and security. The backend must be fully functional and testable via tools like Postman or cURL before the frontend development begins.

C.2 Checkpoints

- ☐ **Project Scaffolding:** Initialize Gradle, Spring Boot, and SQLite configuration.
- ☐ **Data Layer:** Implement Spring Data JPA repositories and entity mappings.
- ☐ **Security Implementation:** Build the authentication and authorization logic.
- ☐ **Business Logic:** Develop required CRUD services for all entities.
- ☐ **Design Validation:** Confirm all implemented endpoints and data models match Phase 1/2 specifications.

C.3 Technical Requirements

C.3.1 API Compliance

All developed endpoints **must** match the API Contract defined in Phase 1/2. Any deviation from the agreed-upon JSON structure or endpoint path must be approved by the team before implementation.

C.3.2 Security Standards

- **Authentication:** Ensure all protected routes (Todo/Subtask management) require a valid user session or token.
- **Authorization:** Ensure users can only access, edit, or delete data that belongs to their own account.

C.3.3 Error Handling

The API must return meaningful HTTP status codes:

- `200 OK` / `201 Created`: Successful operations.
- `400 Bad Request`: Validation errors (e.g., missing required fields).
- `401 Unauthorized`: Missing or invalid authentication.
- `403 Forbidden`: Authenticated user attempting to access unauthorized data.
- `404 Not Found`: Resource does not exist.

C.4 Deliverables for Phase 3

By the end of this phase, your team should have:

1. **Functional REST API:** A running Spring Boot application.
2. **Verified Endpoints:** A collection of successful tests (via Postman, cURL, or similar) demonstrating all CRUD operations.
3. **Persisted Data:** Evidence that data persists in the SQLite database after application restarts.

C.5 Moving to Phase 4

Once the backend is stable and the API endpoints are verified, proceed to **Phase 4: Frontend Development**.

C.6 Project Scaffolding

Initialize Gradle, Spring Boot, and SQLite configuration. See deliverable checklist in Section C.

C.7 Data Layer

Implement Spring Data JPA repositories and entity mappings. See deliverable checklist in Section C.

C.8 Security Implementation

Build the authentication and authorization logic. See deliverable checklist in Section C.

C.9 Business Logic

Develop required CRUD services for all entities. See deliverable checklist in Section C.

C.10 Design Validation

Confirm all implemented endpoints and data models match Phase 1/2 specifications. See deliverable checklist in Section C.

C.11 Functional REST API

Document the running Spring Boot application. See deliverable checklist in Section C.

C.12 Verified Endpoints

Record successful tests (via Postman, cURL, or similar) demonstrating all CRUD operations. See deliverable checklist in Section C.

C.13 Persisted Data

Provide evidence that data persists in the SQLite database after application restarts. See deliverable checklist in Section C.

D Phase 4: Frontend Development

D.1 Objective

The goal of this phase is to build a responsive and interactive user interface. This phase focuses on the "interface" of the application: consuming the Spring Boot API, managing user authentication state via JWT, and creating a seamless user experience for task management.

D.2 Checkpoints

- ☐ **Project Scaffolding:** Initialize Angular and configure environment settings.
- ☐ **Service Layer:** Build Angular services to consume the Spring Boot API.
- ☐ **Auth Integration:** Implement login/register forms and JWT-based state management.
- ☐ **Task Management UI:** Build the dashboard, task creation forms, and subtask nested lists.
- ☐ **Design Validation:** Confirm the UI components and user workflows match the Phase 2 wireframes.

D.3 Technical Requirements

D.3.1 Authentication & JWT Handling

- **Token Storage:** Securely store the JWT (e.g., in `localStorage` or a state management store) upon successful login.
- **Interceptors:** Implement an Angular `HttpInterceptor` to automatically attach the `Authorization: Bearer <token>` header to all outgoing API requests.
- **Route Guards:** Implement `CanActivate` guards to prevent unauthenticated users from accessing the Task Dashboard.
- **Logout Logic:** Implement a clear logout flow that clears the token and redirects the user to the login page.

D.3.2 API Integration

- **Asynchronous Operations:** Use RxJS Observables to handle all API calls to ensure the UI remains responsive during data fetching.
- **Error Handling:** Implement user-friendly error messages (e.g., "Invalid credentials" or "Task not found") based on the HTTP status codes returned by the backend.

D.3.3 User Interface (UI)

- **Responsiveness:** Ensure the layout is functional across different screen sizes.
- **Component Hierarchy:** Follow the component structure defined during the Phase 2 design to maintain code modularity.

D.4 Deliverables for Phase 4

By the end of this phase, your team should have:

1. **Functional Angular Application:** A complete, running frontend integrated with the backend.
2. **End-to-End Workflow:** A user can successfully register, log in, manage todos, and manage subtasks without errors.
3. **Verified UI/UX:** A UI that matches the wireframes and provides a smooth, intuitive user experience.

D.5 Moving to Phase 5

Once the frontend is fully integrated and all user stories are functional, proceed to **Phase 5: Presentation**.

D.6 Project Scaffolding

Initialize Angular and configure environment settings. See deliverable checklist in Section D.

D.7 Service Layer

Build Angular services to consume the Spring Boot API. See deliverable checklist in Section D.

D.8 Auth Integration

Implement login/register forms and JWT-based state management. See deliverable checklist in Section D.

D.9 Task Management UI

Build the dashboard, task creation forms, and subtask nested lists. See deliverable checklist in Section D.

D.10 Design Validation

Confirm the UI components and user workflows match the Phase 2 wireframes. See deliverable checklist in Section D.

D.11 Functional Angular Application

Document the complete, running frontend integrated with the backend. See deliverable checklist in Section D.

D.12 End-to-End Workflow

Demonstrate that a user can successfully register, log in, manage todos, and manage subtasks without errors. See deliverable checklist in Section D.

D.13 Verified UI/UX

Document a UI that matches the wireframes and provides a smooth, intuitive user experience. See deliverable checklist in Section D.

E Phase 5: Presentation

E.1 Objective

The goal of this phase is the formal demonstration of project completion. The team must prove that the application meets the original user stories, functions reliably, and was developed using professional software engineering standards.

E.2 Checkpoints

- ☐ **Functional Demo:** Perform a complete walkthrough of all user stories in a live environment.
- ☐ **Technical Review:** Explain the architectural choices, including the Spring Boot/Angular/JWT integration.
- ☐ **Build Verification:** Demonstrate a clean build and successful application startup from scratch.
- ☐ **Post-Mortem/Reflection:** Briefly summarize the team's Agile process and how blockers were handled.

E.3 Presentation Requirements

E.3.1 The Functional Walkthrough

You must demonstrate the following "Happy Path" scenarios:

- **Onboarding:** A new user registers and successfully logs in.
- **Core Workflow:** A user creates a Todo, adds nested subtasks, edits them, and eventually deletes them.
- **Security Check:** Demonstrate that an unauthenticated user cannot access the dashboard.
- **Persistence:** Show that tasks remain in the system after the application is restarted.

E.3.2 Technical Deep-Dive

Be prepared to answer questions regarding:

- **Data Flow:** How a request moves from an Angular component through the JWT interceptor to the Spring Boot controller and finally to SQLite.
- **Schema Design:** Why you chose your specific relational structure for Todos and Subtasks.
- **Challenges:** How the team utilized Git, resolved merge conflicts, or overcame technical blockers identified during Stand-ups.

E.3.3 Build & Deployment

- **Clean Build:** You must be able to run `./gradlew build` (or equivalent) and `ng build` to prove the project is not running on "magic" local configurations.
- **Startup:** The application must start successfully with a single command or set of commands.

E.4 Success Criteria

A successful presentation is evaluated on:

- **Alignment:** Does the software actually do what the User Stories required?
- **Reliability:** Does the application run smoothly during the demo without unexpected crashes?
- **Communication:** Can the team clearly articulate *how* the application works, not just *that* it works?

- **Professionalism:** Did the team demonstrate a disciplined approach to the SDLC and Agile practices?

E.5 Functional Demo

Perform a complete walkthrough of all user stories in a live environment. See deliverable checklist in Section E.

E.6 Technical Review

Explain the architectural choices, including the Spring Boot/Angular/JWT integration. See deliverable checklist in Section E.

E.7 Build Verification

Demonstrate a clean build and successful application startup from scratch. See deliverable checklist in Section E.

E.8 Post-Mortem/Reflection

Briefly summarize the team's Agile process and how blockers were handled. See deliverable checklist in Section E.

Project Complete.